

BUSCA BINÁRIA

e

CONJUNTOS(SET)

Prof^a Gláucya Boechat

glaucya.boechat@ba.docente.senai.br

Slides adaptados dos prof Rubisley Lemes e
Rodrigo Rocha (IC - UFBA)



BUSCA BINÁRIA (Binary Search)



Busca: definição

Entrada:

vector **v**, com **n** elementos,
elemento **elem** a ser buscado

Saída:

índice (posição) do elemento na lista
-1, se o elemento não for encontrado

Se o elemento aparece várias vezes na lista, qualquer de suas posições é válida.

Busca: exemplo

v:

23	12	74	12	19	22
0	1	2	3	4	5

buscar **23** → 0

buscar **19** → 4

buscar **33** → -1 (não encontrado)

buscar **12** → 1 (ou 3)

Busca binária

- A busca binária (Binary Search) é um algoritmo eficiente para encontrar um elemento em uma lista ordenada.
- A ideia principal é dividir o vetor ao meio repetidamente.
- Em vez de verificar elemento por elemento, o algoritmo elimina metade da lista a cada passo.
- Complexidade:
 - Melhor caso: $O(1)$.
 - Pior caso: $O(\log n)$.

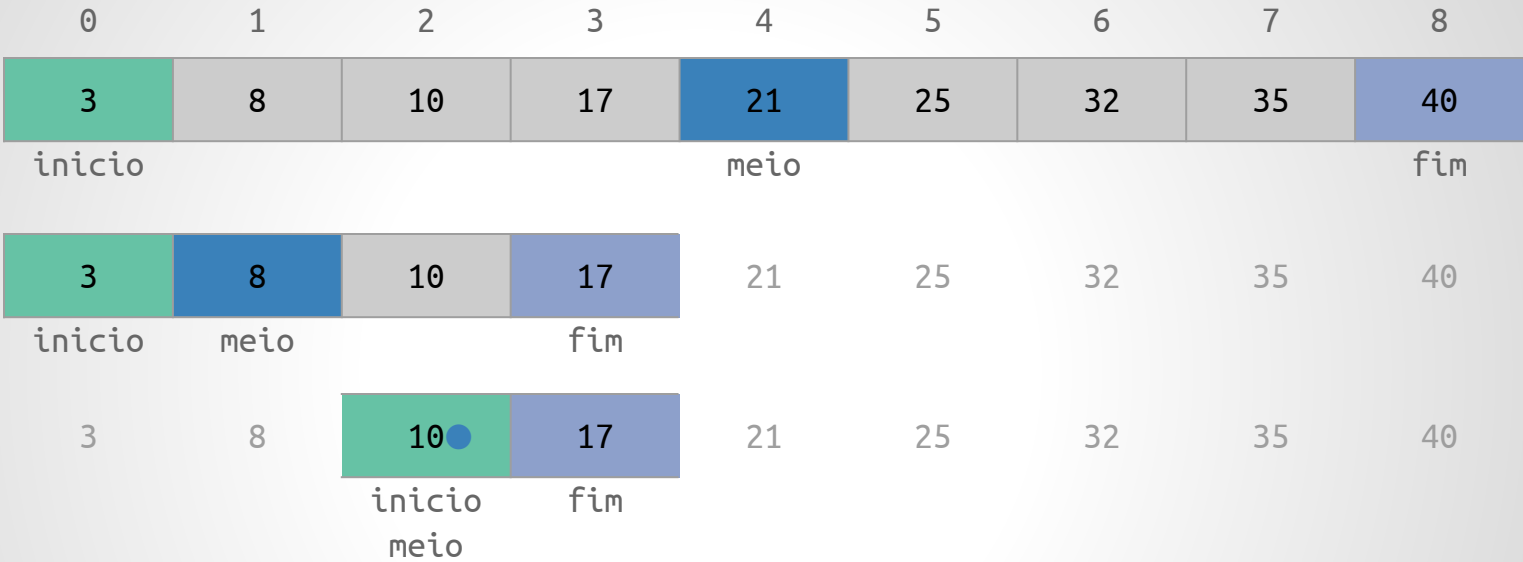
Busca binária

- Quando o vector **v** está ordenado, podemos usar um algoritmo **mais eficiente**: busca binária
- Intuição:
 - Seleciona elemento do meio, **v[meio]** e compara com o elemento buscado, **elem**.
 - Se **elem < v[meio]**, repete considerando os elementos à esquerda;
 - Se **elem > v[meio]**, repete considerando os elementos à direita;
 - Caso contrário, o elemento foi encontrado.

Se o vector **não está ordenado**, o algoritmo de busca binária **não funciona!**

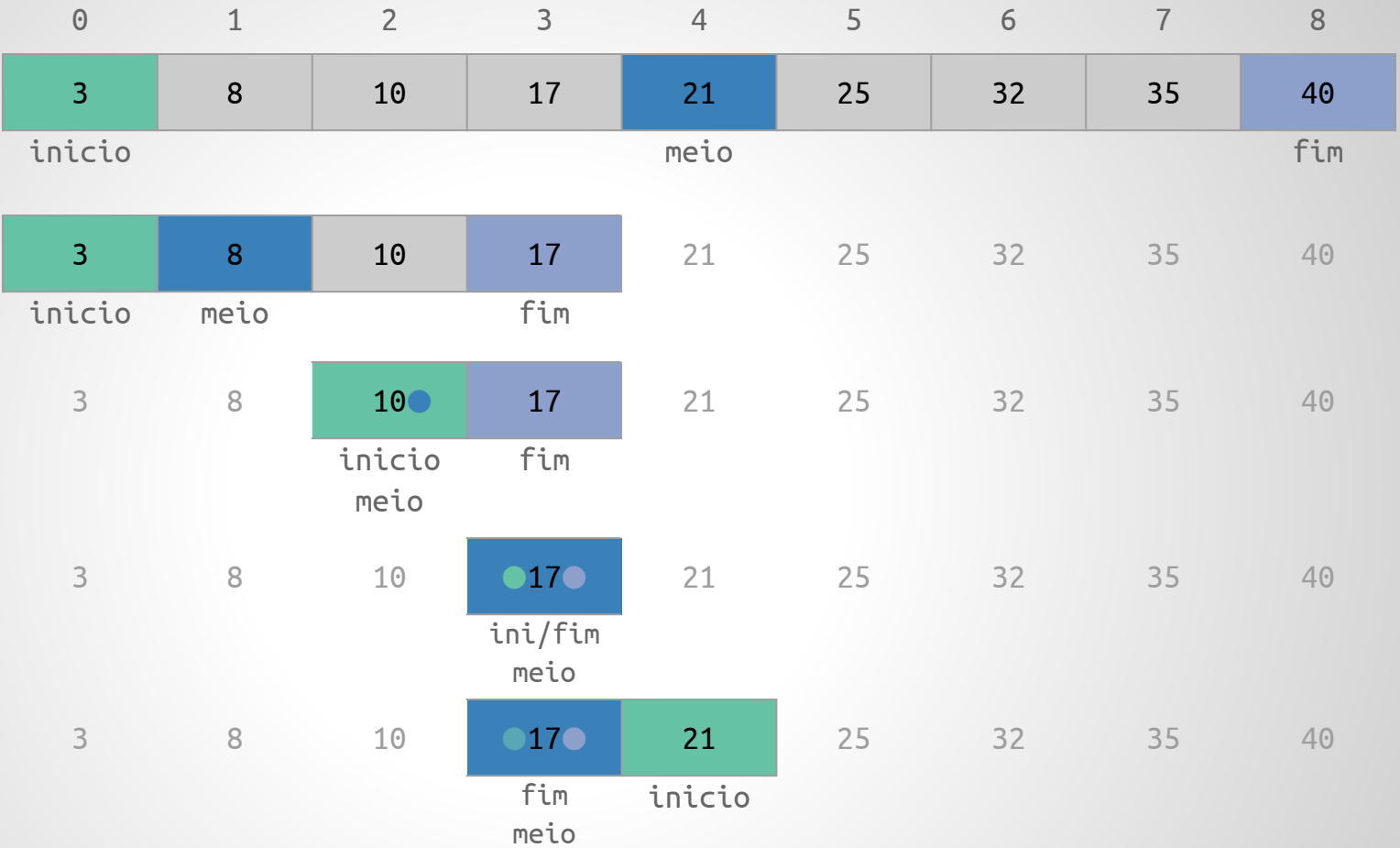
```
BuscaBinaria (A, inicio, fim, x)
  se(inicio ≤ fim)
    m ← (inicio + fim)/2 //calcula o meio
    se(x = A[m])
      retorne m //elemento encontrado
    senão
      se(x > A[m])
        //faz a busca no subvetor à direita
        retorne BuscaBinaria(A, m + 1, fim, x)
      senão
        //faz a busca no subvetor à esquerda
        retorne BuscaBinaria(A, inicio, m - 1, x)
  fim_se
fim_se
senão
  retorne -1 //elemento não encontrado
fim_se
fim
```

Exemplo: buscar **10**



Elemento encontrado na posição **2**

buscar 20



Quando **inicio > fim**, o elemento **não foi encontrado**; **inicio** guarda o local onde ele deveria estar
(ponto de inserção) 9

Busca binária x busca sequencial



Busca binária - Exemplo



<https://pt.khanacademy.org/computer-programming/binary-search-with-selectable-search-key-hit-backspace-and-type/4590599206076416/embedded?embedded?embed=yes&editor=no&width=720>

Função - `binary_search`

- Testa se o valor existe em vector com elementos ordenados.
 - Retorna verdadeiro (true) se **algum elemento** no intervalo [primeiro, último) no vector
 - e false caso contrário.

Binary_search - exemplo

v:

12	12	19	22	23	74
0	1	2	3	4	5

buscar **23** → 1 (true)

buscar **19** → 1 (true)

buscar **33** → 0 (false)

buscar **12** → 1 (true)

Função - binary_search

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    vector<int> v = {23, 12, 74, 12, 19, 22};
    sort (v.begin(), v.end());

    bool r = binary_search(v.begin(), v.end(), 12);
    cout << "12? " << r << endl;
    cout << "23? " << binary_search(v.begin(), v.end(), 23) << endl;
    cout << "33? " << binary_search(v.begin(), v.end(), 33) << endl;
}
```

Saída:

```
12? 1
23? 1
33? 0
```

Função - binary_search (struct)

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
struct pessoa {
    int id;
    string nome;
};
bool cmp(pessoa i, pessoa j) {
    return (i.id < j.id || i.id == j.id && i.nome <
j.nome);
}
int main() { //...
```

```
vector<pessoa> v;
pessoa x;
for (int i=0; i<3 ; i++){
    cin >> x.id >> x.nome;
    v.push_back(x);
}
stable_sort(v.begin(), v.end(), cmp);
pessoa j;
cin >> x.id >> x.nome;
bool r = binary_search(v.begin(), v.end(), x, cmp);
cout << r << endl;
}
```

Entrada
3 Ana
1 Jose
2 Maria
1 Jose

Saída
1

// pode usar uma função de comparação, assim como o sort



AULA - CONJUNTOS



Teoria dos Conjuntos

- Um conjunto é toda e qualquer coleção de elementos.
- Exemplo
 - Conjunto A que seja construído a partir dos números Naturais e que qualquer elemento de A seja maior ou igual a 5 e menor ou igual a 10.

$$A = \{5, 6, 7, 8, 9, 10\}$$

- Como sabemos que qualquer elemento x que pertence a A é um número Natural neste exemplo, podemos representar o conjunto da seguinte maneira:

$$A = \{x \in \mathbb{N} : 5 \leq x \leq 10\}$$

Notação:

$\in \rightarrow$ pertence

$\notin \rightarrow$ não pertence

Teoria dos Conjuntos

- Conjunto vazio

- Existem alguns conjuntos que são representados por uma propriedade onde não é possível gerar elementos.
- Exemplo

$$A = \{x \in \mathbb{N} : 1 < x < 2\}$$

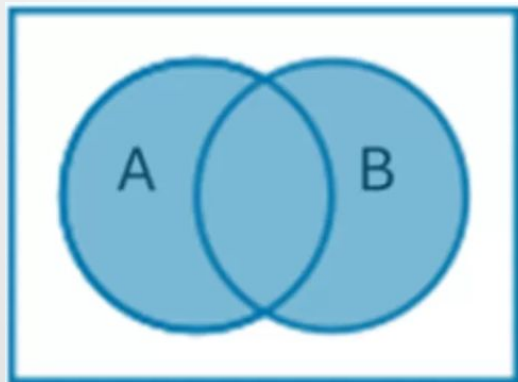
$$A = \emptyset$$

- Qualquer elemento de **A** pertence ao conjunto dos naturais, porém é um absurdo dizer que nos naturais existem números entre 1 e 2, ou seja, em $\mathbb{N}$ não existe o número 1,5, por exemplo. Então, neste caso, dizemos que o conjunto A é vazio.

União de Conjuntos ($\cup$)

- A união de dois conjuntos contém todos os elementos que pertencem a A ou a B.

$$A \cup B = \{x : x \in A \text{ ou } x \in B\}$$



Fonte: https://colaborae.com.br/wp-content/uploads/2024/05/diagramas_de_venn-1.png

A região cinza simboliza a união dos seus respectivos elementos.

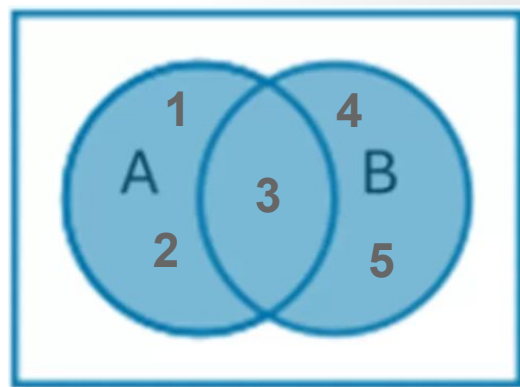
União de Conjuntos ($\cup$)

- Exemplo

- $A = \{1,2,3\}$

- $B = \{3,4,5\}$

- $A \cup B = \{1,2,3,4,5\}$

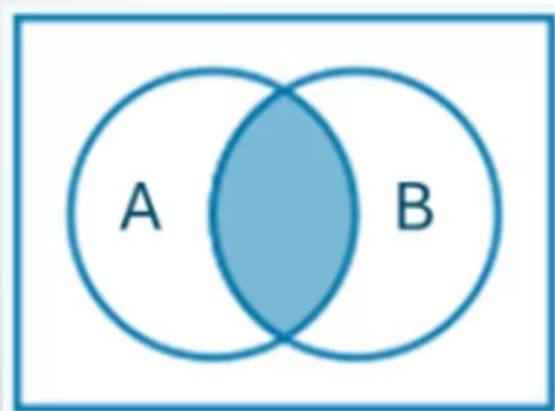


Fonte: https://colaborae.com.br/wp-content/uploads/2024/05/diagramas_de_venn-1.png

Interseção de Conjuntos ($\cap$)

- A interseção contém apenas os elementos comuns aos dois conjuntos.

$$A \cap B = \{x : x \in A \text{ e } x \in B\}$$



Fonte: https://colaborae.com.br/wp-content/uploads/2024/05/diagramas_de_venn-1.png

A região cinza simboliza a união dos seus respectivos elementos.

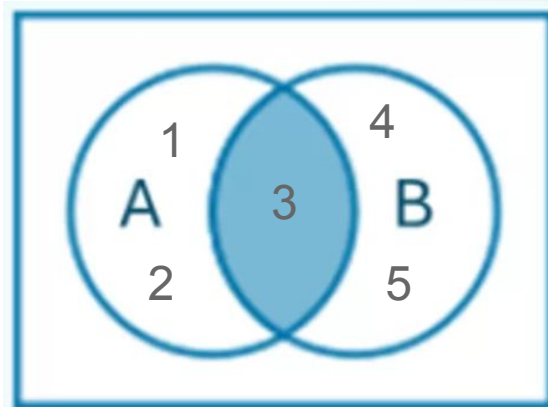
Interseção de Conjuntos ($\cap$)

- Exemplo

- $A = \{1, 2, 3\}$

- $B = \{3, 4, 5\}$

- $A \cap B = \{3\}$

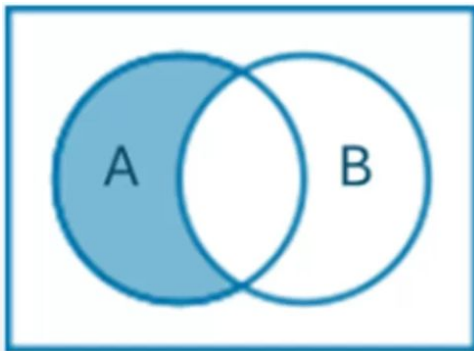


Fonte: https://colaborae.com.br/wp-content/uploads/2024/05/diagramas_de_venn-1.png

Diferença de Conjuntos

- A diferença entre dois conjuntos A e B é o conjunto formado pelos elementos de A que não pertencem a B, formalmente dado por:

$$A - B = \{x : x \in A \text{ e } x \notin B\}$$



Fonte: https://colaborae.com.br/wp-content/uploads/2024/05/diagramas_de_venn-1.png

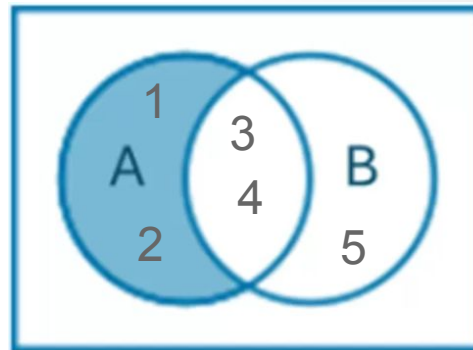
Diferença de Conjuntos

- Exemplo

- $A = \{1, 2, 3, 4\}$

- $B = \{3, 4, 5\}$

- $A - B = \{1, 2\}$

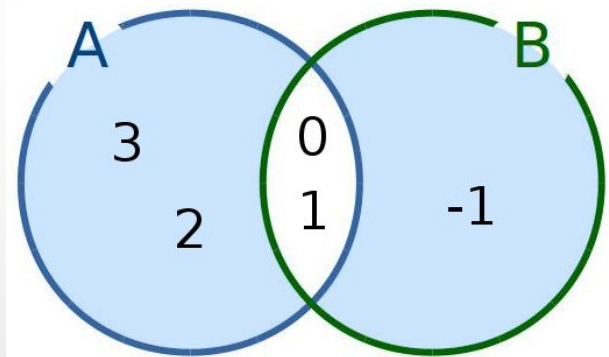


Fonte: https://colaborae.com.br/wp-content/uploads/2024/05/diagramas_de_venn-1.png

Diferença Simétrica de Conjuntos

- A diferença entre dois conjuntos A e B é o conjunto formado pelos elementos que estão em apenas um dos conjuntos.

$$A \square B = (A - B) \cup (B - A)$$



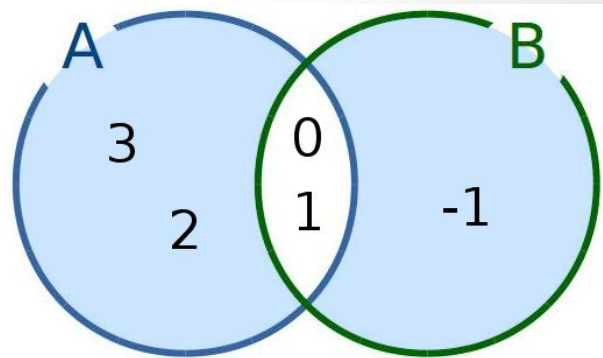
Diferença de Conjuntos

- Exemplo

- $A = \{0, 1, 2, 3\}$

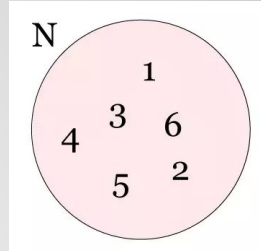
- $B = \{-1, 0, 1\}$

- $A \setminus B = \{-1, 2, 3\}$



<https://autociencia.blogspot.com/2016/08/teoria-dos-conjuntos-diferenca-simetrica.html>

Conjuntos (Set)



- Os conjuntos são contêineres que armazenam elementos exclusivos seguindo uma ordem específica (Classificação).
- Cada elemento deve ser um valor único
- Os elementos são ordenados automaticamente.
- O valor dos elementos em um conjunto não pode ser modificado uma vez inseridos no contêiner (os elementos são sempre constantes),
 - mas podem ser inseridos ou removidos do conjunto.
- Geralmente, a complexidade de tempo de operações como inserção e exclusão no conjunto em C++ é $O(\log n)$

Conjuntos - set

```
#include <iostream>
```

```
#include <set>
```

```
using namespace std; // set -> std
```

```
int main() {
```

```
    set<int> c;
```

```
    for(int i=0; i < 5; i++) {
```

```
        c.insert(i*5);
```

```
    }
```

```
    set<int>::iterator it;
```

```
    for(it = c.begin(); it != c.end(); it++){
```

```
        cout << *it << endl;
```

```
    }
```

Saída:

```
$ g++ ex.cpp -std=c++11
```

```
$ a.out
```

```
0
```

```
5
```

```
10
```

```
15
```

```
20
```

Operações com Conjuntos - União

```
#include <iostream>
#include <set>
#include <algorithm>
using namespace std;
```

```
int main(){
    set<int> A = {1,2,3,4};
    set<int> B = {3,4,5,6};
    set<int> resultado;
    set_union(A.begin(), A.end(),
              B.begin(), B.end(),
              inserter(resultado, resultado.begin()));
    for(int x : resultado)
        cout << x << " ";
}
```

Saída:

\$ g++ ex.cpp -std=c++11

\$ a.out

1 2 3 4 5 6

Operações com Conjuntos - Interseção

```
#include <iostream>
#include <set>
#include <algorithm>
using namespace std;
int main(){
    set<int> A = {1,2,3,4};
    set<int> B = {3,4,5,6};
    set<int> resultado;

    set_intersection(A.begin(), A.end(),
                    B.begin(), B.end(),
                    inserter(resultado, resultado.begin()));
    for(int x : resultado)
        cout << x << " ";
}
```

Saída:

```
$ g++ ex.cpp -std=c++11
```

```
$ a.out
```

```
3 4
```

Operações com Conjuntos - Diferença

```
#include <iostream>
#include <set>
#include <algorithm>
using namespace std;
int main(){
    set<int> A = {1,2,3,4};
    set<int> B = {3,4,5,6};
    set<int> resultado;

    set_difference(A.begin(), A.end(),
                  B.begin(), B.end(),
                  inserter(resultado, resultado.begin()));
    for(int x : resultado)
        cout << x << " ";
}
```

Saída:

```
$ g++ ex.cpp -std=c++11
```

```
$ a.out
```

```
1 2
```

Operações com SET - Diferença Simétrica

```
#include <iostream>
#include <set>
#include <algorithm>
using namespace std;
int main(){
    set<int> A = {1,2,3,4};
    set<int> B = {3,4,5,6};
    set<int> resultado;

    set_symmetric_difference(A.begin(), A.end(),
                             B.begin(), B.end(),
                             inserter(resultado, resultado.begin()));
    for(int x : resultado)
        cout << x << " ";
}
```

Saída:

```
$ g++ ex.cpp -std=c++11
```

```
$ a.out
```

```
1 2 5 6
```


Conjuntos - set

`#include <set>`

Biblioteca necessária para o uso de conjuntos.

`set<TIPO> S;`

TIPO é qualquer tipo da linguagem: int, char, double, vector<float>, ...

`S.clear();`

Remove todos os elementos do conjunto.

`S.insert(ELEMENTO);`

Adiciona ELEMENTO do tipo TIPO no conjunto, **caso ele ainda não exista!**

`S.begin();`

Retorna iterator para o elemento inicial do conjunto.

`S.end();`

Retorna iterator para o fim do conjunto, **e não contém um elemento!**

`S.erase(ELEMENTO);`

Remove ELEMENTO, caso ele existe no conjunto.

`S.erase(it);`

Remove elemento apontado pelo iterator **it**.

`S.find(ELEMENTO);`

Se ELEMENTO existe, retorna um iterator para ele, caso contrário retorna `S.end()`.

`S.count(ELEMENTO);`

Se ELEMENTO existe, retorna um 1, caso contrário retorna 0.

`S.size();`

Retorna a quantidade de elementos do conjunto.

Exemplo - set

```
#include <iostream>
#include <set>
#include <iterator>
using namespace std;

int main() {
    int v[]={ 12,12,37,64,15,40,5,3 };
    set<int> S(v,v+5);    // c1 recebe os 5 primeiros valores exclusivos de v

    set<int>::iterator it;
    for(it = S.begin(); it != S.end(); it++)
        cout << *it << " ";
    cout << "Existe o elemento 12 : " << S.count(12) << endl;

    it = S.find(37);
    it = prev(it); // Retorna um iterator para o elemento anterior de it.
    cout << "Elemento anterior de 37 : " << *it << endl;
    S.erase(it);
    S.erase(12);

    //Retorna um iterator para o elemento posterior de S.begin()
    it = next(S.begin());
    cout << "Segundo elemento:" << *it << endl;
}
```

Saída:

```
$ g++ ex.cpp -std=c++11
```

```
$ a.out
```

```
12 15 37 64
```

```
Existe o elemento 12 : 1
```

```
Elemento anterior de 37 : 15
```

```
Segundo elemento : 64
```

Referências

Binary_search. http://www.cplusplus.com/reference/algorithm/binary_search/

Set <http://www.cplusplus.com/reference/set/set/>